CISS350: Data Structures and Advanced Algorithms
Quiz q10305

Name: eagraham1@cougars.ccis.edu Score:

Here's an example on sorting a vector:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

template < typename T >
std::ostream & operator<<(std::ostream & cout, const std::vector< T > & v)
{
    cout << '{';
    std::string delim = "";
    for (unsigned int i = 0; i < v.size(); ++i)
    {
        cout << delim << v[i]; delim = ", ";
    }
    cout << '}';
    return cout;
}

int main()
{
    std::vector< int > x {5, 3, 1, 2, 4, 0};
    std::cout << x << '\n';
    std::sort(x.begin(), x.end());
    std::cout << x << '\n';
    return 0;

}
```

This sorts the whole vector. (Note that `std::sort` is not stable.) For details on `begin()` and `end()`, see chapter on Containers (on iterators).

Q1. Party planning problem. There are $n = 5$ friends living on a street at address $0, 5, 3, 9, 3$. The address are integers from 0 to $N - 1$ where N is huge, say $N = 1000000$. The houses are on a straight line.

```
            P
            P
P                   P               P
+---+---+---+---+---+---+---+---+---+---+---+---+---+ ...
0   1   2   3   4   5   6   7   8   9   10  11  12  13 ...
```

(P marks an address where a friend lives there.) The distance between 0 and 1 is 1, the distance between 1 and 2 is 1, etc. The friends are planning a party and they have decided to hold it at the house at address 5. The total distance travelled by everyone to 5 is $5 + 2 + 2 + 0 + 4 = 13$.

```
            P------>
            P------>
P------------------>P<--------------P
+---+---+---+---+---+---+---+---+---+---+---+---+---+ ...
0   1   2   3   4   5   6   7   8   9   10  11  12  13 ...
```

Find "best house", i.e., the one where the total distance travelled is the smallest. Note that the brute force solution (using CISS240 methods) would run in $O(n^2)$. Your algorithm must run with time of $O(n \lg n)$ and a space complexity of $O(1)$. (There's actually a faster algorithm that runs with $O(n)$ time.)
Answer:

```
#include <iostream>
#include <cstdlib>
#include <vector>
#include <algorithm>

int main()
{
    int seed;
    std:cin >> seed;
    srand(seed);
    int n = 1000000;
    std::vector< int > x(n);
    for (int i = 0; i < n; ++i)
    {
        x[i] = rand() % 1000000;
    }
    int best_address = x[0];

    std::sort(x.begin(), x.end());
```

```
    int best_address = x[n / 2];

    std::cout << best_address << '\n';
}
```

Q2. A permutation is a shuffle of values. For instance 1, 3, 5, 6, 4 is a permutation of 3, 1, 4, 6, 5. Complete the following problem. Clearly, your code that sets `is_permutation` to `true` if and only if `u` is a permutation of `v`. This must work for different vectors including vectors of large sizes. You should not assume the values are unique. Your algorithm must run with time of $O(n \lg n)$ and a space complexity of $O(1)$.

Answer:

```
#include <iostream>
#include <cstdlib>
#include <vector>
#include <algorithm>

int main()
{
    std::vector< int > u {1, 3, 5, 6, 4};
    std::vector< int > v {3, 1, 4, 6, 5};
    bool is_permutation = true;

    if(u.size() != v.size())
    {
      is_permutation = false;
    }
    else
    {
      std::sort(u.begin(), u.end());
      std::sort(u.begin(), u.end());

      if(u[i] != v[i])
      {
        is_permutation = false;
        break;
      }
    }

    std::cout << is_permutation << '\n';
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.